- Generally, you should handle exceptions at the middleware level, but you can use exception filters to customize how you handle exceptions for specific actions, controllers, or Razor Pages.
- Result filters run before and after an `IActionResult` is executed. You can use them to control how the action result is executed or to completely change the action result that will be executed.
- All filters can short-circuit the pipeline by setting a response. This generally prevents the request progressing further in the filter pipeline, but the exact behavior varies with the type of filter that is short-circuited.
- Result filters aren't executed when you short-circuit the pipeline using authorization, resource, or exception filters. You can ensure that result filters also run for these short-circuit cases by implementing a result filter as `IAlwaysRunResultFilter` or `IAsyncAlwaysRunResultFilter`.
- You can use `ServiceFilterAttribute` and `TypeFilterAttribute` to allow dependency injection in your custom filters. `ServiceFilterAttribute` requires that you register your filter and all its dependencies with the DI container, whereas `TypeFilterAttribute` requires only that the filter's dependencies have been registered.